P1: -ALU

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity alu4bit is
    port (
        n1 : in STD_LOGIC_VECTOR(3 downto 0);
        n2 : in STD_LOGIC_VECTOR(3 downto 0);
        op : in STD_LOGIC_VECTOR(2 downto 0);
        res : out STD_LOGIC_VECTOR(3 downto 0);
        flag : out STD_LOGIC
    );
end alu4bit;

architecture Behavioral of alu4bit is

begin

    process(n1, n2, op)
        variable temp : STD_LOGIC_VECTOR(4 downto 0);
    begin

        flag <= '0';

        case op is

            -- Addition
            when "000" =>
```

```
    temp := ('0' & n1) + n2;
    res <= temp(3 downto 0);
    flag <= temp(4);

-- Subtraction
when "001" =>
    if (n1 > n2) then
        res <= n1 - n2;
    else
        res <= n2 - n1;
    end if;

-- AND
when "010" =>
    res <= n1 and n2;

-- OR
when "011" =>
    res <= n1 or n2;

-- XOR
when "100" =>
    res <= n1 xor n2;

-- NOR
when "101" =>
    res <= n1 nor n2;

-- NOT
when "110" =>
```

```
            res <= not n1;

        -- Default
        when others =>
            res <= n2;

    end case;

end process;

end Behavioral;
```

P2: -SHIFT REGISTER

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_unsigned.ALL;

entity shiftreg is

    Port (
        clk  : in STD_LOGIC;
        rst  : in STD_LOGIC;
        load : in STD_LOGIC;
        si   : in STD_LOGIC;

        pi   : in STD_LOGIC_VECTOR(3 downto 0);
        po   : out STD_LOGIC_VECTOR(3 downto 0);

        mode : in STD_LOGIC_VECTOR(1 downto 0);
        so   : out STD_LOGIC
    );

end shiftreg;


architecture Behavioral of shiftreg is

    signal clkout  : STD_LOGIC;
    signal temp    : STD_LOGIC_VECTOR(3 downto 0);

    signal counter : integer range 0 to 49999999;

begin


    -- CLOCK DIVIDER
    process(clk, rst)
    begin

        if(rst = '1') then

            clkout  <= '0';
            counter <= 0;

        elsif(clk'event and clk = '1') then
```

```
        if(counter = 49999999) then

            clkout  <= not clkout;
            counter <= 0;

        else

            counter <= counter + 1;

        end if;

    end if;

end process;


-- SHIFT REGISTER
process(clkout, rst)
begin

    if(rst = '1') then

        so <= '0';
        po <= "0000";
        temp <= "0000";

    elsif(clkout'event and clkout = '1') then

        case mode is


            -- PIPO
            when "00" =>

                po <= pi;
                so <= '0';


            -- SISO
            when "01" =>

                temp(0) <= si;
                temp(1) <= temp(0);
                temp(2) <= temp(1);
```

```
        temp(3) <= temp(2);

        so <= temp(3);

        po <= "0000";


    -- PISO
    when "10" =>

        if(load = '1') then

            temp <= pi;

        else

            temp(2) <= temp(3);
            temp(1) <= temp(2);
            temp(0) <= temp(1);
            temp(3) <= '0';

            so <= temp(0);

        end if;


    -- SIPO
    when others =>

        if(load = '0') then

            temp(0) <= si;
            temp(1) <= temp(0);
            temp(2) <= temp(1);
            temp(3) <= temp(2);

        else

            po <= temp;

        end if;


end case;
```

```
        end if;

    end process;

end Behavioral;
```

P3: -MOD

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity mod6_counter is
    Port (
        clk : in STD_LOGIC;
        rst : in STD_LOGIC;
        q : out STD_LOGIC_VECTOR(3 downto 0)
    );
end mod6_counter;

architecture Behavioral of mod6_counter is

    signal count : unsigned(3 downto 0) := "0000";
    signal clk_count : integer range 0 to 49999999 := 0;

begin

    process(clk, rst)
    begin

        if rst = '1' then
            count <= "0000";
            clk_count <= 0;

        elsif rising_edge(clk) then
```

```
        if clk_count = 49999999 then
            clk_count <= 0;

            if count = 5 then
                count <= "0000";
            else
                count <= count + 1;
            end if;

        else
            clk_count <= clk_count + 1;
        end if;

    end if;

  end process;

  q <= std_logic_vector(count);

end Behavioral;
```

TESTBENCH:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mod6_counter_tb is
end mod6_counter_tb;
```

```
architecture Behavioral of mod6_counter_tb is

    signal clk : STD_LOGIC := '0';
    signal rst : STD_LOGIC := '1';
    signal q : STD_LOGIC_VECTOR(3 downto 0);

begin

    -- Connect testbench to MOD-6 counter
    DUT : entity work.mod6_counter
        port map (
            clk => clk,
            rst => rst,
            q => q
        );

    -- Generate clock
    clk <= not clk after 5 ns;

    process
    begin

        -- Reset
        rst <= '1';
        wait for 20 ns;

        -- Start counting
        rst <= '0';

        -- Let counter run
```

```
        wait for 500 ns;

        wait;

    end process;

end Behavioral;
```

P4: FIFO

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity FIFO is
    Port (
        clk     : in  STD_LOGIC;
        rst     : in  STD_LOGIC;
        wren    : in  STD_LOGIC;
        rden    : in  STD_LOGIC;
        empty   : out STD_LOGIC;
        full    : out STD_LOGIC;
        dataout : out STD_LOGIC_VECTOR(7 downto 0)
    );
end FIFO;

architecture Behavioral of FIFO is

    -- Clock divider signals
    signal clkout  : STD_LOGIC := '0';
    signal counter : integer range 0 to 49999999 := 0;

    -- FIFO pointers and count
    signal rdptr : integer range 0 to 7 := 0;
    signal wrptr : integer range 0 to 7 := 0;
    signal count : integer range 0 to 8 := 0;

    -- FIFO memory: 8 locations, 8 bits each
    type fifo1 is array (0 to 7) of
```

```
        STD_LOGIC_VECTOR(7 downto 0);

    signal memory : fifo1 := (others => (others => '0'));

    -- Input data
    type datain1 is array (0 to 7) of
        STD_LOGIC_VECTOR(7 downto 0);

    signal data : datain1 := (
        "10101010",
        "10111011",
        "10010101",
        "11110000",
        "00001111",
        "10100101",
        "11110101",
        "01100110"
    );

begin

    -- Clock frequency division: 100 MHz to 1 Hz
    process(clk, rst)
    begin
        if rst = '1' then
            clkout  <= '0';
            counter <= 0;

        elsif rising_edge(clk) then
            if counter = 49999999 then
```

```
            clkout  <= not clkout;
            counter <= 0;
        else
            counter <= counter + 1;
        end if;
    end if;
end process;


-- Actual FIFO process using the divided clock
process(clkout, rst)
begin
    if rst = '1' then

        for i in 0 to 7 loop
            memory(i) <= (others => '0');
        end loop;

        rdptr   <= 0;
        wrptr   <= 0;
        count   <= 0;
        empty   <= '1';
        full    <= '0';
        dataout <= (others => '0');

    elsif rising_edge(clkout) then

        -- Write operation
        if wren = '1' and rden = '0' then
```

```
    if count < 8 then
        memory(wrptr) <= data(wrptr);

        if wrptr = 7 then
            wrptr <= 0;
        else
            wrptr <= wrptr + 1;
        end if;

        count <= count + 1;
        empty <= '0';

        if count = 7 then
            full <= '1';
        else
            full <= '0';
        end if;
    end if;

-- Read operation
elsif rden = '1' and wren = '0' then

    if count > 0 then
        dataout <= memory(rdptr);

        if rdptr = 7 then
            rdptr <= 0;
        else
            rdptr <= rdptr + 1;
        end if;
```

```
                count <= count - 1;
                full <= '0';

                if count = 1 then
                    empty <= '1';
                else
                    empty <= '0';
                end if;
            end if;

        end if;

    end if;
  end process;

end Behavioral;
```

P5: -KEYPAD: -

File 1:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity PmodKYPD is
   Port (
      clk : in STD_LOGIC;
      JA  : inout STD_LOGIC_VECTOR(7 downto 0);
      -- PmodKYPD is designed to be connected to JA

      an  : out STD_LOGIC_VECTOR(3 downto 0);
      -- Controls which position of the seven-segment display is active

      seg : out STD_LOGIC_VECTOR(6 downto 0)
      -- Digit to display on the seven-segment display
   );
end PmodKYPD;

architecture Behavioral of PmodKYPD is

   component Decoder is
      Port (
         clk       : in STD_LOGIC;
         Row       : in STD_LOGIC_VECTOR(3 downto 0);
         Col       : out STD_LOGIC_VECTOR(3 downto 0);
         DecodeOut : out STD_LOGIC_VECTOR(3 downto 0)
      );
   end component;

   component DisplayController is
      Port (
         DispVal : in STD_LOGIC_VECTOR(3 downto 0);
         anode   : out STD_LOGIC_VECTOR(3 downto 0);
         segOut  : out STD_LOGIC_VECTOR(6 downto 0)
      );
   end component;

   signal Decode : STD_LOGIC_VECTOR(3 downto 0);
```

```
begin

    C0 : Decoder
        port map (
            clk       => clk,
            Row       => JA(7 downto 4),
            Col       => JA(3 downto 0),
            DecodeOut => Decode
        );

    C1 : DisplayController
        port map (
            DispVal => Decode,
            anode   => an,
            segOut  => seg
        );

end Behavioral;
```

File 2:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity Decoder is
   Port (
      clk       : in STD_LOGIC;
      Row       : in STD_LOGIC_VECTOR(3 downto 0);
      Col       : out STD_LOGIC_VECTOR(3 downto 0);
      DecodeOut : out STD_LOGIC_VECTOR(3 downto 0)
   );
end Decoder;

architecture Behavioral of Decoder is

   signal sclk : STD_LOGIC_VECTOR(19 downto 0);

begin

   process(clk)
   begin
      if clk'event and clk = '1' then

         -- 1ms
         if sclk = "00011000011010100000" then

            -- C1
            Col <= "0111";
            sclk <= sclk + 1;

         -- Check row pins
         elsif sclk = "00011000011010101000" then

            -- R1
            if Row = "0111" then
               DecodeOut <= "0001"; -- 1

            -- R2
            elsif Row = "1011" then
               DecodeOut <= "0100"; -- 4

            -- R3
```

```
    elsif Row = "1101" then
        DecodeOut <= "0111"; -- 7

    -- R4
    elsif Row = "1110" then
        DecodeOut <= "0000"; -- 0
    end if;

    sclk <= sclk + 1;

-- 2ms
elsif sclk = "00110000110101000000" then

    -- C2
    Col <= "1011";
    sclk <= sclk + 1;

-- Check row pins
elsif sclk = "00110000110101001000" then

    -- R1
    if Row = "0111" then
        DecodeOut <= "0010"; -- 2

    -- R2
    elsif Row = "1011" then
        DecodeOut <= "0101"; -- 5

    -- R3
    elsif Row = "1101" then
        DecodeOut <= "1000"; -- 8

    -- R4
    elsif Row = "1110" then
        DecodeOut <= "1111"; -- F
    end if;

    sclk <= sclk + 1;

-- 3ms
elsif sclk = "01001001001111100000" then

    -- C3
    Col <= "1101";
    sclk <= sclk + 1;
```

```
-- Check row pins
elsif sclk = "01001001001111101000" then

    -- R1
    if Row = "0111" then
        DecodeOut <= "0011"; -- 3

    -- R2
    elsif Row = "1011" then
        DecodeOut <= "0110"; -- 6

    -- R3
    elsif Row = "1101" then
        DecodeOut <= "1001"; -- 9

    -- R4
    elsif Row = "1110" then
        DecodeOut <= "1110"; -- E
    end if;

    sclk <= sclk + 1;

-- 4ms
elsif sclk = "01100001101010000000" then

    -- C4
    Col <= "1110";
    sclk <= sclk + 1;

-- Check row pins
elsif sclk = "01100001101010001000" then

    -- R1
    if Row = "0111" then
        DecodeOut <= "1010"; -- A

    -- R2
    elsif Row = "1011" then
        DecodeOut <= "1011"; -- B

    -- R3
    elsif Row = "1101" then
        DecodeOut <= "1100"; -- C
```

```
            -- R4
            elsif Row = "1110" then
               DecodeOut <= "1101"; -- D
            end if;

            sclk <= "00000000000000000000";

         else
            sclk <= sclk + 1;

         end if;
      end if;
   end process;

end Behavioral;
```

File 3:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity DisplayController is
    Port (
        -- Output from the Decoder
        DispVal : in STD_LOGIC_VECTOR(3 downto 0);

        -- Controls the display digits
        anode : out STD_LOGIC_VECTOR(3 downto 0);

        -- Controls which digit to display
        segOut : out STD_LOGIC_VECTOR(6 downto 0)
    );
end DisplayController;

architecture Behavioral of DisplayController is

begin

    -- Only display the leftmost digit
    anode <= "1110";

    with DispVal select
        segOut <=
            "1000000" when "0000", -- 0
            "1111001" when "0001", -- 1
            "0100100" when "0010", -- 2
            "0110000" when "0011", -- 3
            "0011001" when "0100", -- 4
            "0010010" when "0101", -- 5
            "0000010" when "0110", -- 6
            "1111000" when "0111", -- 7
            "0000000" when "1000", -- 8
            "0010000" when "1001", -- 9
            "0001000" when "1010", -- A
            "0000011" when "1011", -- B
            "1000110" when "1100", -- C
            "0100001" when "1101", -- D
            "0000110" when "1110", -- E
            "0001110" when "1111", -- F
```

```
        "0111111" when others;

end Behavioral;
```